LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Name: G Ganikhasri
SRN: PES1UG24AM098
Section: B

Hypothesis-based fuzz testing of the Spam Scoring Filter

Objective

To apply property-based fuzz testing using the Hypothesis library, identify input-validation bugs in the supplied Python application, and improve its robustness against unexpected or invalid inputs.

Application under test

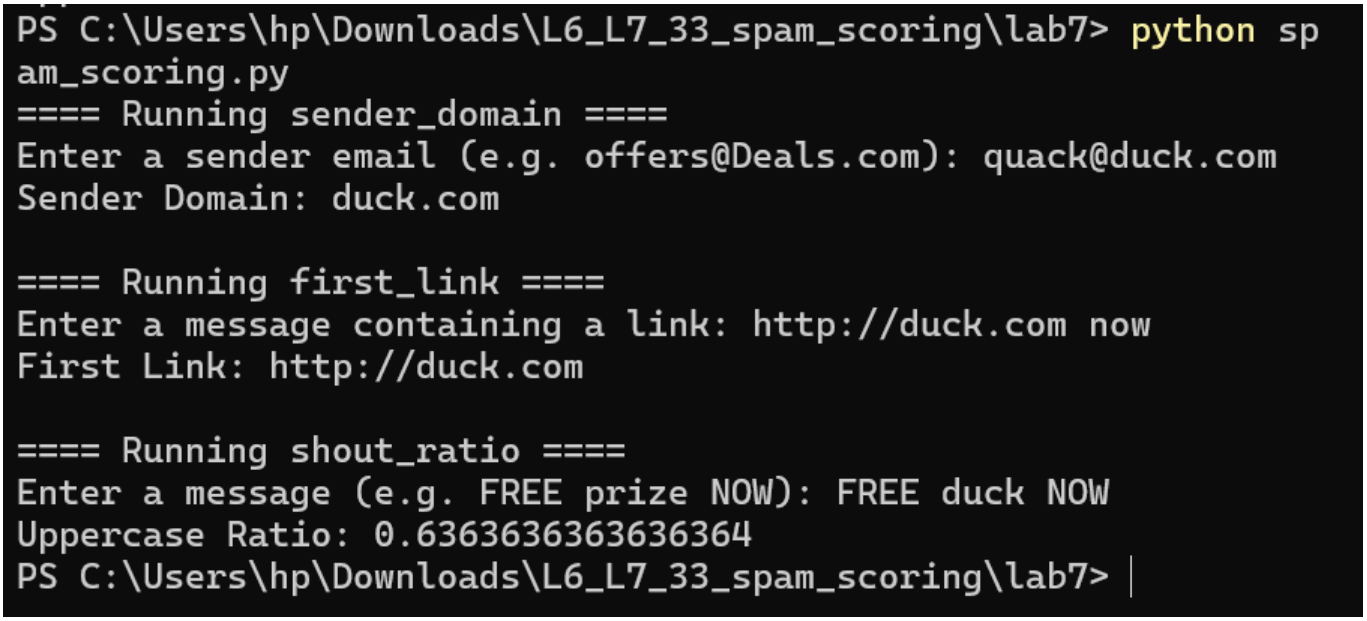
The supplied `spam_scoring.py` module contains three functions: `sender_domain()`, `first_link()`, and `shout_ratio()`. The tests are in `test_spam_scoring.py`.

Tool used

Python 3.11.8, pytest 9.2.0, and Hypothesis 6.170.15.

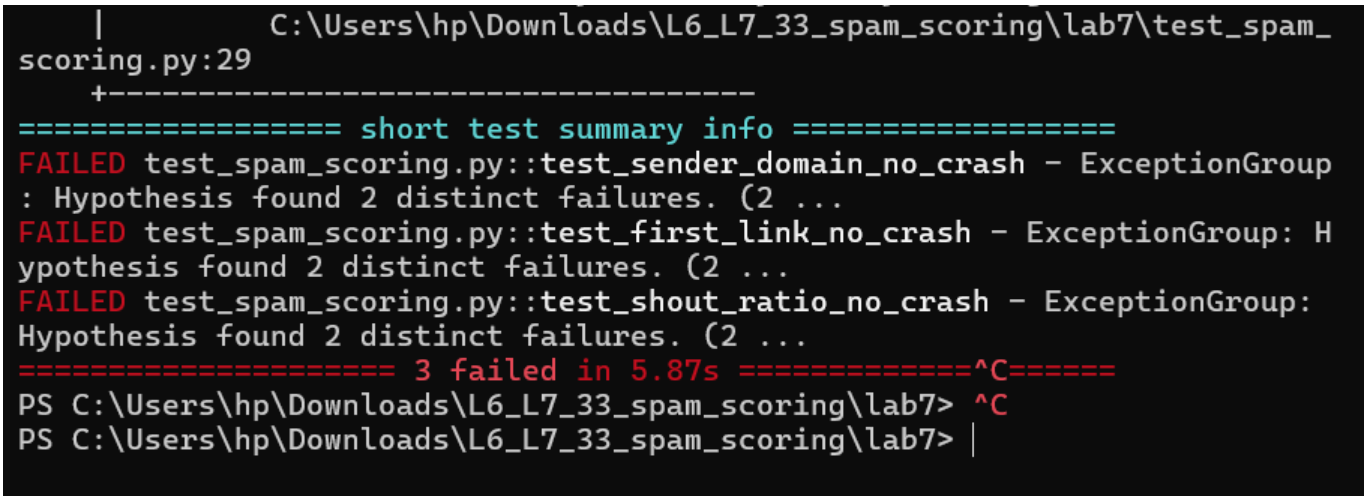
1. Initial application execution

The original application was executed with valid sample inputs. It successfully extracted the sender domain, returned the first link, and calculated the uppercase-letter ratio.



2. Hypothesis fuzzing – failures in the original application

The initial property-based tests failed for all three functions. Hypothesis minimized the generated inputs and exposed failures involving empty strings and None values. The observed exception types included IndexError, AttributeError, ValueError, ZeroDivisionError, and TypeError.



Issues identified

• **sender_domain()**: assumes the input is a string containing an @ symbol and a non-empty domain.
• **first_link()**: assumes the input is a string containing the substring http.
• **shout_ratio()**: divides by the number of alphabetic characters without handling the zero-letter case.

3. Defensive fixes applied

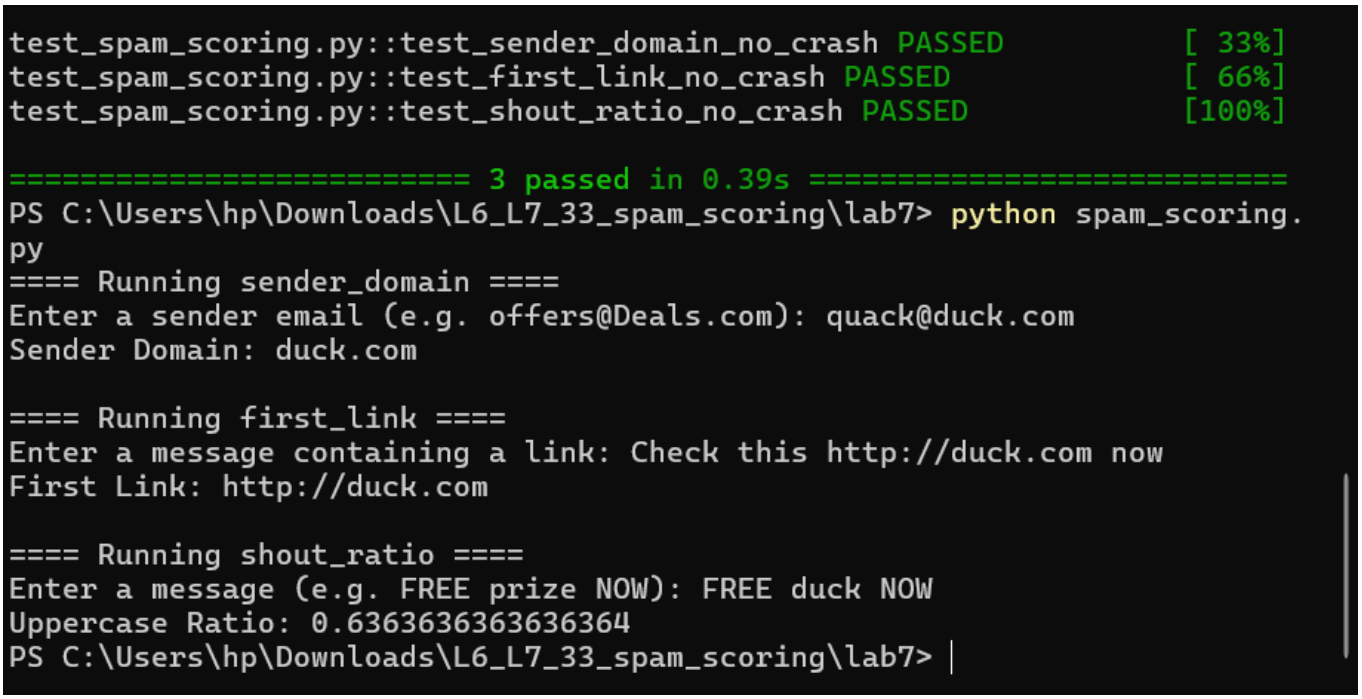
The application was hardened while retaining the three original function names and the interactive main program.

• **sender_domain()** now validates that the input is a string, requires exactly one @ symbol, and rejects an empty local part or domain. Invalid inputs return an empty string.
• **first_link()** now validates the input type and uses find() to detect a missing http substring. Invalid inputs or messages without a link return an empty string.
• **shout_ratio()** now validates the input type and returns 0.0 when there are no alphabetic characters, avoiding division by zero.

The same three Hypothesis no-crash tests were then rerun against the corrected application.

4. Final application execution

The corrected application ran successfully with the sample sender, link-containing message, and uppercase message. It returned duck.com, http://duck.com, and an uppercase ratio of 0.6363636363636364.



5. Final Hypothesis test results

All three property-based tests passed after the fixes: sender_domain_no_crash, first_link_no_crash, and shout_ratio_no_crash. The captured run reports 3 passed.

```
PS C:\Users\hp\Downloads\L6_L7_33_spam_scoring\lab7> python -m pytest -v
test_spam_scoring.py
========================= test session starts =========================
platform win32 -- Python 3.11.8, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\h
p\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\hp\Downloads\L6_L7_33_spam_scoring\lab7
plugins: anyio-4.16.0, hypothesis-6.170.15, cov-7.1.0
collected 3 items

test_spam_scoring.py::test_sender_domain_no_crash PASSED          [ 33%]
test_spam_scoring.py::test_first_link_no_crash PASSED             [ 66%]
test_spam_scoring.py::test_shout_ratio_no_crash PASSED            [100%]

========================== 3 passed in 0.36s ==========================
PS C:\Users\hp\Downloads\L6_L7_33_spam_scoring\lab7>
```

Conclusion

Hypothesis successfully exposed input-validation and edge-case failures in all three functions. After adding defensive checks, the same fuzz tests passed, and the sample application executed successfully. This demonstrates how property-based fuzz testing can reveal unexpected inputs that ordinary valid-input demonstrations may not cover.